WORKSHOP

Enterprise Agentic AI:

Architecting Autonomous Java Systems for Production

Hands-On Workshop • 90 minutes

Java | QUARKUS | LangChain4j by QUARKUS

SECURE BY DESIGN
Zero trust, mTLS, policies & guardrails

BUILT FOR PERFORMANCE
Quarkus speed, scale-to-zero, native

OBSERVABLE END-TO-END
OpenTelemetry traces & metrics

BUILT FOR AUTONOMY
Multi-agent, A2A, event-driven

THE PROBLEM

QUARKUS | LangChain4j

The Problem: Apex Systems NOC

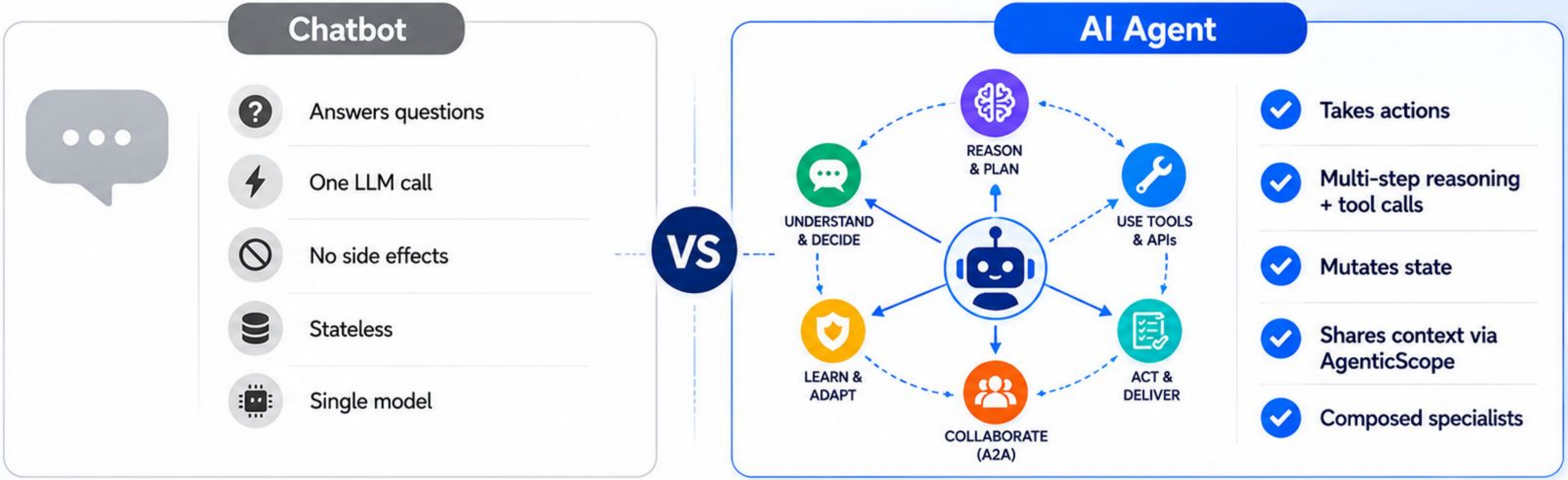
Manual triage is failing

- P1 outage misclassified as P3 → **4+ hours, $150k lost**
- Payment incident routed to wrong team → **20 engineer-hours** wasted
- No audit trail → **compliance gap**

Too many tickets. Too little context. Too much at stake.

It's time to move from manual to intelligent.

Why Agentic AI, Not a Chatbot



Agentic capabilities you will build

REASON
Understand context, plan next steps, and solve complex problems.

ACT
Use tools and APIs, execute actions, and update systems.

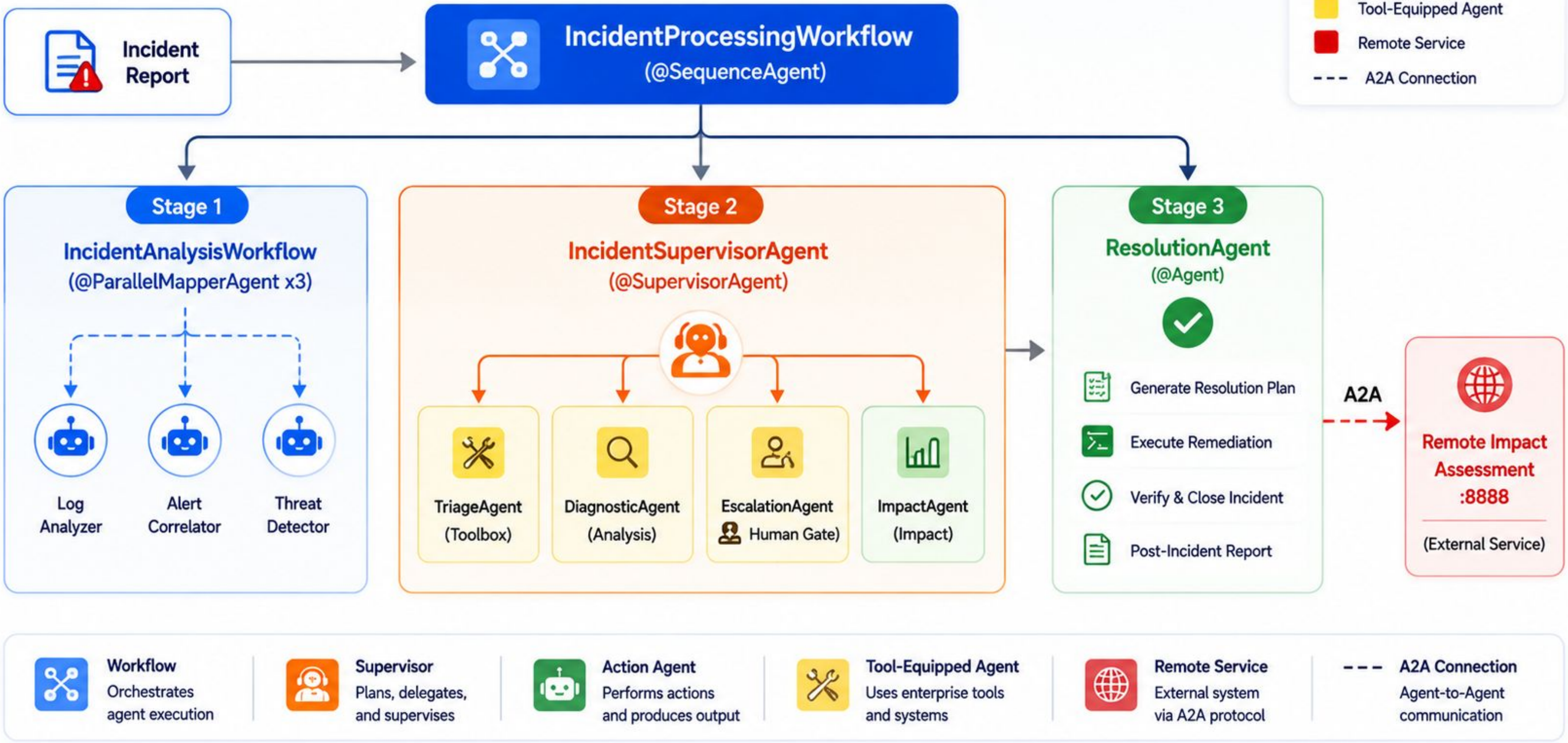
COLLABORATE
Work with other agents and humans across systems (A2A).

PAUSE FOR HUMANS
Seek approval, confirm decisions, and stay in control.

SCALE
Scale out agents and workloads with Kubernetes and event-driven architectures.

What You Will Build

A multi-agent incident management pipeline



Your Learning Path — 8 Exercises

From first agent to enterprise-ready multi-agent system

Goal: Build, run, and govern agentic systems you can trust in production.

1. Agent + Tool
Sam, NOC analyst

Build your first agent with a tool. Analyze an incident using real data.

2. Policy as Prompt
Chris, Ops lead

Encode routing policies and guardrails as prompts and test their impact.

3. Parallel Agents
Chris, Ops lead

Run multiple specialized agents in parallel to speed up analysis.

4. Supervisor Orchestration
Priya, IT service mgr

Use a supervisor agent to orchestrate specialists and make decisions.

5. AI Governance
Jordan, Platform engineer

Apply governance controls, model allow-lists, and content filters.

6. Human Gate + Tracing
Alex, Compliance officer

Add human approval checkpoints and trace every step for auditability.

7. Remote Agents A2A
Riley, SRE lead

Integrate external agents via A2A protocol for impact assessment.

8. Quality Loop (bonus)
Jordan, Platform engineer

Add feedback loops to continuously improve agent performance.

Exercises 1–4: code-along
Build core agents and workflows together.

Exercise 5: AI governance
Learn how to govern agent behavior safely.

Exercises 6–7: run & explore
Add controls, integrate remote agents, and observe.

Exercise 8: bonus
Improve quality with continuous feedback loops.

The Technology Stack

A modern, open, and enterprise-ready foundation for building agentic AI systems in Java.

AGENTS.md

Token-efficient context file, cross-tool standard

- Rules & Guardrails
- Shared Context
- Consistent Behavior
- Lightweight & Efficient

OpenCode CLI

SDLC partner: plan → code → test → secure

- Plan & Scaffold
- Code & Refactor
- Test & Validate
- Secure & Review

Quarkus LangChain4j

Declarative @Agent interfaces, workflows, A2A

- @Agent Model
- Workflows & Orchestration
- A2A Protocol
- Tools & Integrations

Quarkus

Build-time agent validation, fast startup, hot reload

- Ultra-Fast Startup
- Secure by Design
- Observability Ready
- High Performance

How They Connect

AGENTS.md — Provides rules, context & project knowledge

↓

OpenCode CLI — Uses context to help you build, test & secure

↓

Quarkus LangChain4j — Implements agents, workflows & A2A

↓

Quarkus — Runs everything fast, secure & observable

Built for Production

Performance — Fast startup, low memory footprint, and high throughput.

Security — Secure by design with policy enforcement and human oversight.

Observability — End-to-end tracing, metrics, and cost visibility.

Autonomy — Composed agents that reason, act, and adapt at scale.

The Declarative @Agent Model

You declare WHAT the agent does — Quarkus generates HOW it works.

TriageAgent (Java Interface)

```java
public interface TriageAgent {

    @SystemMessage
    "You are a senior NOC triage analyst.
     Classify incidents, gather evidence,
     and recommend next actions."

    @UserMessage
    """
    Incident ID: {id}
    Summary: {summarrroy}
    Source: {source}
    Severity guess: {severityGuess}
    """

    @Agent(
      outputKey = "triageResult",
      description = "Structured triage
           result with recommended actions"
    )

    @ToolBox(TriageTool.class)
    TriageResult triage(IncidentInput input);
}
```

@SystemMessage Role & policy

@UserMessage Structured input template

@Agent outputKey + description

@ToolBox Enterprise tool access

LLM Decides
Agent reasons and plans the next step

Tool Executes
Agent calls the appropriate tool from the toolbox

State Changes
Tools update systems and databases transactionally

TriageTool (@ApplicationScoped)

```java
@ApplicationScoped
public class TriageTool {

    @Tool("searchIncidents")
    @Description("Search similar
     incidents and patterns")
    public List<Incident> searchIncidents(
      @ToolParam("query") String query,
      @ToolParam("limit") int limit) {
      ...
    }

    @Tool("updateIncident")
    @Transactional
    @Description("Update incident record
     with triage results")
    public void updateIncident(
      @ToolParam("id") String id,
      @ToolParam("result") TriageResult result) {
      // JPA entity update
      ...
    }
}
```

@Tool LLM-callable method

@Transactional JPA mutation

You declare WHAT — Quarkus generates HOW.
No classes. No boilerplate. Just interfaces, annotations, and tools.

Fast to Build | Type Safe | Maintainable | Production Ready

Agent Interface · Tool Class · Execution Flow · - - - Invocation Flow

Enterprise Controls Built In

Governed. Observable. Human-centered.

AI Governance

Policy and context guardrails

AGENTS.md

1. Use Approved APIs Only
2. No Data Exfiltration
3. PII Must Be Masked
4. Human Approval for P1
5. Log All Actions
6. Least Privilege Access
7. Verify Before Change
8. Cost-Aware Responses
9. Secure by Default
10. Explain Your Decisions

- 10 project rules enforced from turn 1
- Guardrail refusal for hallucinated APIs
- Token-efficient context loading across tools

Human-in-the-Loop

Control where it matters

Human Approval Required — P1 Escalation

Agent EscalationAgent requests approval to escalate incident INC-54821 to the Payments Team.

Reason
High customer impact detected. Payment service degraded for 18 minutes. Estimated financial impact: $150K+.

Recommended Action
Escalate to Payments On-Call (Team: PAY-OPS)

✕ Reject | ✓ Approve

Analyze → Escalate (P1) → Human Gate → Execute → Audit Log

- P1 escalations require human approval
- Named checkpoints in agent pipeline
- Audit trail for compliance

Observability

See everything. Understand anything.

- OpenTelemetry gen_ai spans
- Trace every LLM call and tool invocation
- FinOps cost tracking per request

Secure by design. Observable end-to-end. Human oversight where it matters.

- Governed from day one
- Humans in control of critical decisions
- Full visibility across agents and tools
- Built for scale, compliance, and trust

Let's Build!

Spin up the over-engineered helpdesk server and explore the demo.

Java | QUARKUS | LangChain4j

Quick Start

```
# 1. Clone the repository
$ git clone https://github.com/danieloh30/agentic-ai-java-workshop.git
$ cd agentc-ai-java-workshop

# 2. Start the helpdesk server
$ ./mvnw clean quarkus:dev
[INFO] Quarkus started in 2.341s. Listening on: http://localhost:8080

# 3. Open the dashboard
$ open http://localhost:8080
```

This app is intentionally over-engineered with MCP.
Let's see why simplicity wins.

Quarkus Lightning fast Java runtime

LangChain4j Agentic workflows made simple

MCP SDK Model Context Protocol

OpenAI Powering intelligent decisions

Incident Dashboard

Monitor and manage helpdesk incidents

8 Total Incidents
Across all priorities

2 P1 - Critical Needs immediate attention	3 P2 - High High priority incidents	2 P3 - Medium Requires attention	1 P4 - Low Low priority incidents

ID	TITLE	PRIORITY	STATUS	ASSIGNEE	UPDATED
INC-54821	Payment service degraded	P1 Critical	OPEN	Priya S.	2m ago
INC-54813	Checkout timeouts	P1 Critical	OPEN	Alex R.	7m ago
INC-54805	Search API slow responses	P2 High	IN PROGRESS	Jordan K.	15m ago
INC-54799	Email notifications failing	P2 High	OPEN	Riley M.	28m ago
INC-54788	User profile not updating	P2 High	IN PROGRESS	Chris L.	45m ago
INC-54770	Report generation slow	P3 Medium	OPEN	Sam D.	1h ago
INC-54766	Password reset delay	P3 Medium	IN PROGRESS	Taylor W.	2h ago
INC-54740	UI alignment on mobile	P4 Low	OPEN	Jamie P.	3h ago

Open localhost:8080 — Exercise 1 starts now.

List all tickets and tell me which one sounds most urgent.

Get the full source code

github.com/danieloh30/agentic-ai-java-workshop.git